Let's see what that looks like. Implement the missing color and temperature methods inside the Coffee class:

02-running-specs/11/spec/coffee_spec.rb

```
def color
  ingredients.include?(:milk) ? :light : :dark
end

def temperature
  ingredients.include?(:milk) ? 190.0 : 205.0
end
```

Now, try rerunning your specs:

```
$ rspec
..FF

Failures:

  1) A cup of coffee with milk is light in color FIXED
     Expected pending 'Color not implemented yet' to fail. No error was ↩
     raised.
     # ./spec/coffee_spec.rb:42

  2) A cup of coffee with milk is cooler than 200 degrees Fahrenheit FIXED
     Expected pending 'Temperature not implemented yet' to fail. No error ↩
     was raised.
     # ./spec/coffee_spec.rb:47

Finished in 0.00293 seconds (files took 0.08214 seconds to load)
4 examples, 2 failures

Failed examples:

rspec ./spec/coffee_spec.rb:42 # A cup of coffee with milk is light in color
rspec ./spec/coffee_spec.rb:47 # A cup of coffee with milk is cooler than ↩
200 degrees Fahrenheit
```

RSpec has marked the test suite as failing, because we have examples marked as pending that are actually implemented now. When you remove the pending bits, the entire suite will pass.

If you *really* don't want the body of the spec to run at all, you can use skip in place of pending. Or you can use xit, which is a temporary annotation like fit except that it skips the example instead of focusing it.

Use pending to Flag Errors in Third-Party Code



If your spec is failing because of a bug in a dependency, mark it with pending and the ticket ID from their bug tracker; for example, pending 'waiting for a fix for Hitchhiker's Guide bug #42'. When you later update to a version containing the fix, RSpec will tell you.